

An Elementary Introduction to Language Models and Transformers for Statisticians and Probabilists

Sabyasachi Chatterjee

1 Language Models and the Statistical Estimation Problem

Large Language Models seem to have changed the world in the last few years. In this article, we will discuss the basic ideas underlying a language model. In particular, we aim to describe the transformer architecture at a high level. I have been trained in Probability and Statistics and I have written this article so that it is easy to follow for people familiar with probability and statistics.

When we enter a prompt in Chatgpt, how does it give us an answer? Fundamentally, the basic problem it solves is to generate the next word. Once it generates the next word, then the problem becomes of generating the next word and so on. It is hard to believe that solving this problem of generating a good next word makes these language models so powerful in answering a wide variety of questions.

Language Models operate on words and sometimes parts of words, punctuation marks, spaces and other symbols. Each of these fundamental units is called a token.

The fundamental task facing a language model is

Given the previous tokens in a document, predict the next token.

Language Models do this at each step by creating a probability distribution on the set of all tokens and then sampling from it.

For example, after observing

(The, cat),

the model could assign high probability to

sat

being the next token.

Similarly, after observing

(The, capital, of, France, is),

the model should assign high probability to

Paris.

Predicting the next word may seem like a narrow objective. However, in order to perform this task well, a model presumably must learn grammar, semantics, world knowledge, reasoning patterns, and many other regularities of language. Modern language models obtain many of their remarkable capabilities by learning to solve this single prediction problem. It is truly amazing that this is true.

1.1 Probabilistic Formulation of a Language Model

The remarkable first step is to model language as a stochastic process. Let us imagine there is a probability distribution for language; that is for a sequence of words/tokens

$$(X_1, \dots, X_T, X_{T+1})$$

they have a joint distribution.

If we can learn this joint distribution, then we can use the conditional probability

$$P(X_t \mid X_1, \dots, X_{t-1})$$

and sample from it to generate the next token/word.

By the chain rule of probability,

$$P(X_1, \dots, X_T) = \prod_{t=1}^T P(X_t \mid X_1, \dots, X_{t-1}).$$

Therefore, learning the entire joint distribution is equivalent to learning all of the conditional distributions

$$P(X_t \mid X_1, \dots, X_{t-1}).$$

1.2 Statistical Estimation Problem

Now the question arises, how do we learn these conditional probabilities. The unknown object can be seen as a function that takes a sequence of previous tokens and returns a probability distribution for the next token:

$$(X_1, \dots, X_{t-1}) \mapsto P(X_t \in \cdot \mid X_1, \dots, X_{t-1}).$$

Equivalently, given a prefix

$$(X_1, \dots, X_{t-1}),$$

the function outputs probabilities

$$p(v) = P(X_t = v \mid X_1, \dots, X_{t-1}), \quad v \in \mathcal{V}.$$

The fundamental object we seek to estimate from data is a sequence of maps from a prefix of length $t - 1$ to a probability distribution over the set of tokens, where t ranges from 1 to a very large number. This is a hard, seemingly impossible non parametric estimation problem.

1.3 Training Data

Suppose we have access to a large collection of text documents, such as books, websites, scientific articles, source code, Wikipedia pages, and other written material as is available on the internet.

Each document is converted into a sequence of tokens. As mentioned before, a token may correspond to a word, part of a word, punctuation, or some other basic linguistic unit. For example, the sentence

The cat sat on the mat.

might be represented as

(The, cat, sat, on, the, mat, .).

More generally, a document is represented as a sequence

$$(X_1, \dots, X_T),$$

where each X_t takes values in a finite vocabulary

$$\mathcal{V} = \{1, \dots, V\}.$$

The length T may vary from document to document.

The training data consist of many such documents

$$D_1, \dots, D_n,$$

where

$$D_i = (X_1^{(i)}, \dots, X_{T_i}^{(i)}).$$

Thus, the training data are a large collection of separate sequences.

1.4 Parametric Model and Maximum Likelihood (on Steroids)

As we will see, eventually we will design a parametric statistical model via the transformer architecture.

What does a parametric statistical model mean? This means we have modelled conditional probabilities

$$P_\theta(X_t = v \mid X_1, \dots, X_{t-1}), \quad v \in \mathcal{V},$$

for prefixes $X_1, \dots, X_{t-1}$ of any length and any token v , parametrized by a fixed but very large (in billions and trillions in LLM's) number of parameters. Conceptually, this is just a basic parametric statistical model but now with number of parameters extremely large. As in basic parametric models, once we have data, we want to estimate/learn the parameters. As is taught in basic statistics classes, one fundamental method is the method of maximum likelihood that is used here too.

Consider the training sentence

(The, cat, sat, on, the, mat).

From this single sentence we obtain several prediction tasks:

The $\longrightarrow$ cat,

(The, cat) $\longrightarrow$ sat,

(The, cat, sat) $\longrightarrow$ on,

and so on.

Suppose the model assigns probability

$$P_{\theta}(X_t \mid X_1, \dots, X_{t-1})$$

to the correct next token. The negative log-likelihood loss for that prediction is

$$-\log P_{\theta}(X_t \mid X_1, \dots, X_{t-1}).$$

Summing over all positions and all training documents gives the objective function

$$L(\theta) = - \sum_{i=1}^n \sum_{t=1}^{T_i} \log P_{\theta} \left(X_t^{(i)} \mid X_1^{(i)}, \dots, X_{t-1}^{(i)} \right).$$

Training consists of minimizing this loss, or equivalently maximizing the likelihood of the observed corpus.

Thus, despite the enormous scale of modern language models, the underlying statistical principle is simply maximum likelihood estimation.

One important point to keep in mind is that the probability model is parametrized by a fixed (albeit humungous) number of parameters; irrespective of the number of conditioned variables. We will elaborate on this more later.

Let us recap the basic statistical estimation problem then.

1. What do we want to estimate?

A sequence of functions that maps a prefix of tokens to a probability distribution over the next token.

2. What are the observations?

The observations are token sequences extracted from a large collection of text documents.

3. How is the model trained?

By maximum likelihood estimation, equivalently minimizing the total negative log-likelihood over the training corpus.

The role of a transformer is simply to provide a powerful parameterized family of functions capable of representing these conditional probability distributions.

2 Transformers

Let us forget conditional probability distributions now; we will come back to it later. Let us now talk about what transformers are. **Fundamentally, it is a parametrized class of functions from sequences to sequences of arbitrary length.** Transformers use the so called attention mechanism and neural network architectures to construct mappings designed for processing sequential data such as text, audio, and time series. They form the foundation of modern large language models and have become one of the most important developments in deep learning.

To explain transformers, first we need to understand that language models do not directly operate on words. Rather, words are first embedded into an embedding space which might have dimensions several hundreds to thousands. The embedding is learnt from data; as in word2vec, and apparently this embedding space can have semantic structure. We will come back to how to

do this later. For now, let us assume that we can embed words into a high dimensional euclidean space in which words having similar meanings are somehow closer to each other.

Here we can write a section first on n gram models, the problem with exploding number of terms and then say the great idea is that words can be captured by a lower dimensional space

Consider the two sentences

‘I deposited money in the bank.’

and

‘The boat reached the bank of the river.’

Although the word ‘bank’ appears in both sentences, its meaning differs. To continue and write the next sentence, a language model has to understand the sentence and the context. It has to presumably understand that bank means two different things in the two sentences. However, this is not captured by the word embedding process as one word is typically mapped to only one vector; e.g, bank will be mapped to a single vector in both the sentences.

The next great idea here is therefore to transform the word embeddings in both the sentences to another set of embeddings/vectors which now would be able to capture that bank is a financial institution in the first sentence and bank refers to the side of a river in the second sentence. A transformer attempts to do exactly this, by hopefully producing contextual representations in which the representation of ‘bank’ depends on the surrounding words.

Suppose we are given a sequence of vectors

$$\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$$

where each vector $\mathbf{x}_i \in \mathbb{R}^d$ represents one token or element of the sequence. The goal of a transformer is to convert these input vectors into richer contextual representations

$$\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_n$$

where each output vector incorporates information from the entire sequence. Thus, the representation of a word becomes dependent on its context.

2.1 Can I have your Attention please?

How to produce such contextual representations? The idea is to simply do **weighted averaging based on a similarity measure** which has come to be known as attention. The nomenclature is always so fancy (and sometimes misleading/noninformative/confusing) in ML.

So we have a sequence of words/tokens $x_1, \dots, x_n \in \mathbb{R}^d$. Let us denote the matrix X whose i th row is the token x_i^T written as a row.

The idea now is to create a similarity score for words i and j , let us denote this by s_{ij} . Then we define *attention* weights

$$w_{ij} = \frac{\exp(s_{ij})}{\sum_j \exp(s_{ij})}.$$

In ML, this is called a *softmax* operation. Then we can transform each word x_i into $y_i = \sum_j w_{ij} x_j$.

Now the question comes as to what similarity score do we take? Perhaps the first thing that comes to mind is to take inner products, that is define $s_{ij} = x_i^T x_j$. This leads to the following transformation which we can compactly write in matrix notation.

$$Y = \text{Softmax}(XX^T)X. \tag{1}$$

Here the Softmax operation means exponentiating every entry and then normalizing each row so that row wise we have probability vectors.

Here is where things get a little more interesting. As it stands, this is just a fixed transformation/function from sequences to sequences. To fit training data, we would like to have a whole class of transformations implementing this idea of producing contextual representations in this way of weighted averaging. Another issue could be that in constructing the similarity score, each coordinate (called feature in ML) of the words play an equal role. We would like more flexibility in using different features to construct the similarity score. In other words, maybe we can use other notions of similarities or inner products?

One way to do this is to first do a linear transformation of the variables XU where the entries of the matrix U consists of learnable parameters and then use the transformation as in (1). This will then lead to the transformation

$$Y = \text{Softmax}(XU[XU]^T)XU. \quad (2)$$

This is now parametrized by a matrix which gives us a flexible model. However, this is perhaps not flexible/rich enough. One possible issue could be that before doing the softmax operation, the similarity matrix $(XU[XU]^T)$ is symmetric. Presumably, one might want the attention to be *nonsymmetric*. For example, in the sentence "The boat reached the bank of the river", it makes sense that bank should put significant weight on river but river need not put an equal amount of weight on bank.

So then the question is how can we make the similarity score non symmetric and make the model more flexible? To do this, the developers of transformers came up with an ingenious idea; the idea of separating the three components in the transformation (2), denoted by Q, K and V below.

$$Y = \text{Softmax}(\underbrace{XU}_Q \underbrace{[XU]^T}_K) \underbrace{XU}_V.$$

So we will have three different matrices $W^{(q)}, W^{(k)}, W^{(v)}$ which will give three different linear transformations which will represent the query, key and value vectors for any word/token x . Let us define

$$Q = XW^{(q)}, \quad K = XW^{(k)}, \quad V = XW^{(v)}.$$

We can now generalize (2) to obtain a class of transformations

$$Y = \text{Softmax}(QK^T)V = \text{Softmax}(XW^{(q)}[XW^{(k)}]^T) XW^{(v)}. \quad (3)$$

There are a lot of matrices running around. For once, let us be explicit about the dimensions of these matrices. Here, $X \in \mathbb{R}^{n \times d}$ denotes the matrix of token representations, where n is the sequence length and d is the embedding dimension. The matrices

$$W^{(q)} \in \mathbb{R}^{d \times d_k}, \quad W^{(k)} \in \mathbb{R}^{d \times d_k}, \quad W^{(v)} \in \mathbb{R}^{d \times d_v}$$

are learnable parameter matrices. Consequently,

$$Q, K \in \mathbb{R}^{n \times d_k}, \quad V \in \mathbb{R}^{n \times d_v}.$$

The matrix product QK^T therefore has dimension $\mathbb{R}^{n \times n}$ and contains similarity scores between all pairs of tokens in the sequence. After applying the softmax operation row-wise, the resulting attention matrix still has dimension $n \times n$. Finally, the output of the attention layer satisfies

$$Y \in \mathbb{R}^{n \times d_v}.$$

The terminology of *queries*, *keys*, and *values* comes originally from information retrieval systems and databases. In such systems, a user submits a *query*; this query is compared with a collection of *keys* in order to determine which stored items are most relevant; the associated *values* are then retrieved.

The transformer tries to mirror the same idea. Each token representation produces three different vectors:

- a *query vector*, representing the kind of information this token is currently looking for,
- a *key vector*, representing the type of information contained in the token,
- a *value vector*, representing the actual information that will be passed along if the token is deemed relevant.

To understand the intuition, consider the sentence

$$\text{“The movie was exciting because the ending was surprising.”} \quad (4)$$

Suppose we are updating the representation of the word “exciting.” To understand why the movie was exciting, it may be useful to attend to the word “surprising.” The query vector associated with “exciting” asks: *which other words contain information relevant for describing the emotional character of the movie?*

Each other token provides a key vector describing what type of information it contains. The word “surprising” may produce a key vector strongly aligned with the query from “exciting,” leading to a large attention score. Its value vector is then incorporated into the updated representation of “exciting.”

Thus, queries determine *what information is being sought*, keys determine *which tokens are relevant*, and values determine *what information is actually communicated*. The use of three separate learned linear transformations allows the model to represent these three roles differently.

We make one final refinement to the transformation (3). We need to rescale the inner products in QK^T before applying the softmax. Otherwise, here we are taking inner products of vectors of length d_k which is typically large, and hence we will typically get large numbers. Gradients of the softmax function become exponentially small for inputs of high magnitude and hence training becomes slow. We simply divide each inner product by $\sqrt{d_k}$. This is apparently because if all the query and key elements were independent random variables with mean 0 and variance 1 then the variance of the dot product would be d_k ; hence dividing by $\sqrt{d_k}$ will rescale it to have unit variance.

Finally, we are ready to write down the class of transformations given by what is called a single *attention head*.

$$Y = \text{Softmax}\left(\frac{1}{\sqrt{d_k}}QK^T\right)V = \text{Softmax}\left(\frac{1}{\sqrt{d_k}}XW^{(q)}[XW^{(k)}]^T\right)XW^{(v)}. \quad (5)$$

Remark 1. Note that the dimensions of the three matrices do not involve n ; so in principle the class of transformations in (5) can act on sequences of arbitrary length.

2.2 Multi-Head Attention

The transformation in (5) is called a single *attention head*. However, it is plausible that a single notion of similarity is not sufficient to capture all the different relationships present in language. For example, one attention head may learn grammatical relationships, another may focus on semantic similarity, while another may learn certain long-range dependencies.

The idea of *multi-head attention* is therefore to construct multiple attention heads in parallel. For the h -th head, we introduce separate parameter matrices

$$W_h^{(q)}, \quad W_h^{(k)}, \quad W_h^{(v)}. \quad (6)$$

This produces outputs

$$Y_h = \text{Softmax} \left(\frac{1}{\sqrt{d_k}} Q_h K_h^T \right) V_h. \quad (7)$$

Suppose we use H different heads. The outputs are concatenated together and then linearly transformed using another learnable matrix $W^{(o)}$:

$$Y = \text{Concat}(Y_1, \dots, Y_H) W^{(o)}. \quad (8)$$

The matrix $W^{(o)} \in \mathbb{R}^{Hd_v \times d}$ is again a learnable parameter matrix and is supposed to combine the information coming from the different heads.

Thus, a single multi-head attention layer defines a rich class of transformations from sequences to sequences. Importantly, all these transformations can still operate on sequences of arbitrary length because none of the parameter dimensions depend on the sequence length.

2.3 Feedforward Nonlinearity and Transformer Layers

The attention mechanism is one component of a transformer layer. After the attention step, each token representation is passed through the same nonlinear transformation independently.

More precisely, if

$$Y = \begin{bmatrix} y_1^T \\ y_2^T \\ \vdots \\ y_n^T \end{bmatrix}, \quad (9)$$

then we apply the same nonlinear map f to every token:

$$z_i = f(y_i), \quad i = 1, \dots, n. \quad (10)$$

Since the same function f is applied independently to each token, this transformation also naturally works for sequences of arbitrary length.

Why is nonlinearity important? We can see that each of the transformed representations still lie in the linear span of the original word representations. This might be a significant restriction. The nonlinearity allows the model to represent far more complicated functions and interactions.

For example, consider the sentence

$$\text{“The movie was not good.”} \quad (11)$$

The meaning of this sentence is not obtained simply by averaging the meanings of the individual words. The word “not” fundamentally changes the sentiment. Capturing such interactions requires nonlinear transformations. **this example is not clear to me.**

In practice, the nonlinear map f is typically implemented using a small feedforward neural network applied separately to each token. Usually this is a two-layer neural network of the form

$$f(y) = W_2\sigma(W_1y + b_1) + b_2, \quad (12)$$

where σ is a nonlinear activation function such as ReLU. **good idea to talk about dimensions here**

Combining the multi-head attention mechanism with this feedforward nonlinearity gives the full class of transformations corresponding to one transformer layer.

In practice, transformers contain many such layers stacked one after another. The output sequence from one layer becomes the input sequence for the next layer.

Why are multiple layers useful? A rough intuition is that early layers may learn local or syntactic relationships, while deeper layers may gradually construct more abstract semantic representations. In other words, just like neural networks with many layers become powerfully expressive, so too is perhaps the case here.

For example, in the sentence

$$\text{“The animal did not cross the street because it was too tired.”} \quad (13)$$

a shallow layer may identify that “it” refers to a previous noun, while deeper layers may infer that “it” refers specifically to “the animal” rather than “the street” because only the animal can plausibly be tired.

Finally, let us summarize the learnable parameters in a transformer layer.

For each attention head we have matrices

$$W_h^{(q)}, \quad W_h^{(k)}, \quad W_h^{(v)}. \quad (14)$$

In addition, we have the output matrix

$$W^{(o)}, \quad (15)$$

and the parameters of the feedforward neural network

$$W_1, W_2, b_1, b_2. \quad (16)$$

If the embedding dimension is d and the sequence length is n , then the dominant computational cost comes from matrix multiplications of size roughly $n \times d$ by $d \times d$ matrices. The resulting computational complexity of one transformer layer is approximately

$$O(nd^2). \quad (17)$$

check the above line

To summarize, transformers define a supposedly highly flexible family of functions from sequences to sequences, parametrized by a large number of learnable parameters and capable of processing sequences of arbitrary length.

the transformers mapping is completely defined, next come back to generating text, add positional encoding, explain how probabilities are created from the last layer and how tokens are samples, finally end with describing word embedding.

3 Back to Generating the Next Word

3.1 Positional Encoding

There is one important ingredient missing from the transformer architecture as described so far.

Recall that the input to the transformer is a sequence of token representations

$$x_1, x_2, \dots, x_n. \quad (18)$$

The attention mechanism computes similarities between tokens and then forms weighted averages. However, nowhere in the construction have we explicitly told the model which token is first, which token is second, and so on.

This is problematic because the order of words in a sentence clearly matters. For example,

$$\text{“The dog chased the cat.”} \quad (19)$$

and

$$\text{“The cat chased the dog.”} \quad (20)$$

contain exactly the same words but have very different meanings.

In fact, one can show mathematically that the transformer map defined so far is *permutation equivariant*. Roughly speaking, if we permute the order of the input tokens, then the output tokens are permuted in exactly the same way. Thus, the transformer as currently defined knows which words are present, but it has no notion of their locations within the sentence.

To overcome this problem, transformers incorporate explicit information about position.

The basic idea is simple. Suppose that each token has an embedding vector

$$e_t \in \mathbb{R}^d. \quad (21)$$

We construct another vector

$$p_t \in \mathbb{R}^d \quad (22)$$

which depends only on the position t of the token in the sequence. The actual input to the transformer is then

$$x_t = e_t + p_t. \quad (23)$$

Thus two occurrences of the same word appearing at different locations will receive different vector representations.

The original transformer paper of [?] uses a deterministic sinusoidal positional encoding. If d denotes the embedding dimension, then for each position t we define

$$p_t(2k) = \sin\left(\frac{t}{10000^{2k/d}}\right), \quad (24)$$

and

$$p_t(2k+1) = \cos\left(\frac{t}{10000^{2k/d}}\right). \quad (25)$$

Thus the coordinates of p_t consist of sine and cosine waves of many different frequencies.

To understand the intuition, imagine that we observe several clocks that tick at different speeds. A single clock does not uniquely determine the time because its readings repeat periodically. However, if we observe a large collection of clocks running at different rates, then the combined collection of readings almost uniquely identifies the time.

The sinusoidal positional encoding works in essentially the same way. Each coordinate records the value of a sinusoidal wave at position t , but the waves oscillate at different frequencies. Some coordinates vary rapidly with position while others vary much more slowly. The resulting vector

$$p_t = (\sin(\omega_1 t), \cos(\omega_1 t), \sin(\omega_2 t), \cos(\omega_2 t), \dots) \quad (26)$$

acts as a high-dimensional fingerprint for the position t .

The use of many frequencies is important. High-frequency coordinates allow the model to distinguish nearby positions, while low-frequency coordinates encode information about larger scales and long-range locations within the sequence.

There is another elegant property of this construction. Recall the trigonometric identity

$$\sin(\omega(t + r)) = \sin(\omega t) \cos(\omega r) + \cos(\omega t) \sin(\omega r). \quad (27)$$

A similar identity holds for the cosine function. Consequently, the positional encoding at location $t + r$ can be obtained from the encoding at location t by applying a linear transformation that depends only on the displacement r .

This means that relative positions are naturally encoded in the representation. For example, a transformer layer can potentially learn concepts such as

“the previous word,” “the next word,” or “three words earlier”

using only linear operations on the positional vectors.

Finally, after positional encodings are added, the transformer is no longer permutation equivariant. The model can now distinguish not only which words appear in a sentence, but also the order in which they appear.

This seemingly simple modification is one of the key ingredients that allows transformers to process sequential data such as language.

3.2 From Hidden Representations to Conditional Probabilities

We now return to the original goal of a language model. Given tokens

$$X_1, \dots, X_t, \quad (28)$$

we want to construct a probability distribution for the next token

$$X_{t+1}. \quad (29)$$

Let the vocabulary be

$$\mathcal{V} = 1, \dots, V. \quad (30)$$

After the token embeddings and positional encodings are passed through the transformer layers, we obtain final contextual representations

$$h_1, \dots, h_t, \quad (31)$$

where

$$h_i \in \mathbb{R}^d. \quad (32)$$

The vectors $(h_1, \dots, h_t)$ may be viewed as the transformer's internal representation of the entire prefix. Each vector corresponds to one token position, but through the repeated attention operations every representation can incorporate information from the surrounding tokens. Thus the representation of a word depends on its context.

It is useful to think of the transformer layers as performing two conceptual tasks. The earlier layers gradually construct increasingly rich contextual representations of the tokens. The final transformer layer then acts as a feature-engineering layer whose goal is to produce representations that are particularly useful for next-token prediction.

In particular, the final layer constructs

$$h_1, \dots, h_t \quad (33)$$

so that the final representation (h_t) contains the information needed to predict the next token. The prediction layer that follows is surprisingly simple: it is merely a linear classifier followed by a softmax transformation.

To convert (h_t) into a probability distribution over the vocabulary, we first apply a linear transformation. Let

$$W^{(p)} \in \mathbb{R}^{d \times V} \quad (34)$$

and

$$b^{(p)} \in \mathbb{R}^V. \quad (35)$$

We define the vector of scores

$$s_t = (h_t)^T W^{(p)} + b^{(p)}. \quad (36)$$

Thus

$$s_t \in \mathbb{R}^V. \quad (37)$$

The (v) -th coordinate $(s_t(v))$ is a score assigned to token (v) as a possible next token. These scores are often called *logits*. They are not probabilities and may be positive, negative, or zero.

Let

$$w_v \in \mathbb{R}^d \quad (38)$$

denote the (v) -th column of $(W^{(p)})$. Then

$$s_t(v) = \langle h_t, w_v \rangle + b_v. \quad (39)$$

Thus each vocabulary word has an associated parameter vector (w_v) . The score assigned to a word is determined by how well its parameter vector aligns with the contextual representation (h_t) .

This admits a natural interpretation. The vector (h_t) may be viewed as a learned feature vector describing the current context, while the vectors (w_v) act as classifier coefficients associated with the various vocabulary words. A word receives a high score when its parameter vector is highly compatible with the current context.

We convert the scores into probabilities using the softmax function:

$$p_t(v) = \frac{\exp(s_t(v))}{\sum_{u \in \mathcal{V}} \exp(s_t(u))}, \quad v \in \mathcal{V}. \quad (40)$$

This produces a probability distribution

$$p_t = (p_t(1), \dots, p_t(V)). \quad (41)$$

The language model is therefore

$$P_\theta(X_{t+1} = v \mid X_1, \dots, X_t) p_t(v). \quad (42)$$

Viewed from a machine learning perspective, the final prediction layer is simply a multinomial logistic regression classifier operating on the learned feature vector (h_t) . The difficult part of the problem is not the classifier itself; rather, it is learning a representation (h_t) for which the correct next token can be identified by such a simple linear rule.

In many modern language models, the output matrix $(W^{(p)})$ is closely related to the embedding matrix used at the input layer. In this case, the vectors (w_v) may be interpreted as learned representations of the vocabulary words themselves. Prediction then amounts to finding which word representation best matches the contextual representation (h_t) produced by the transformer.

For example, after observing

$$\text{“The capital of France is”}, \quad (43)$$

the transformer constructs a contextual representation (h_t) that encodes the information contained in the prefix. The vector associated with the token ‘Paris’ should then have a much larger compatibility score with h_t than the vectors associated with most other words. After the softmax transformation, the model therefore assigns high probability to the token ‘Paris’.

Thus one may think of the transformer as learning a sequence of contextual representations of the input, while the final layer performs a simple linear classification task based on the representation constructed for the last position.

3.3 Training a Transformer Language Model

Suppose a training document consists of the token sequence

$$X_1, \dots, X_T. \quad (44)$$

From this single document we obtain many prediction problems. At position t , the model observes

$$X_1, \dots, X_t \quad (45)$$

and tries to predict

$$X_{t+1}. \quad (46)$$

Therefore the model assigns probability

$$P_\theta(X_{t+1} \mid X_1, \dots, X_t) \quad (47)$$

to the correct next token.

The loss for this prediction is the negative log probability assigned to the correct answer:

$$-\log P_\theta(X_{t+1} \mid X_1, \dots, X_t). \quad (48)$$

For the whole document, the loss is

$$L(\theta) = - \sum_{t=1}^{T-1} \log P_\theta(X_{t+1} \mid X_1, \dots, X_t). \quad (49)$$

For a collection of training documents

$$D_1, \dots, D_N, \quad (50)$$

where

$$D_i = (X_1^{(i)}, \dots, X_{T_i}^{(i)}), \quad (51)$$

the total loss is

$$L(\theta) = - \sum_{i=1}^N \sum_{t=1}^{T_i-1} \log P_\theta \left(X_{t+1}^{(i)} \mid X_1^{(i)}, \dots, X_t^{(i)} \right). \quad (52)$$

Training consists of minimizing this loss over the parameters θ . Equivalently, we are maximizing the likelihood of the observed text corpus.

This is the same maximum likelihood principle that appears throughout statistics. The only difference is that the conditional probability model is now represented by a large transformer network.

There is one important detail. When predicting X_{t+1} , the model should not be allowed to look at X_{t+1} or any later token. Hence, when computing the representation h_t , the attention mechanism is restricted so that token t can only attend to positions

$$1, 2, \dots, t. \quad (53)$$

In other words, attention weights on future positions are set to zero. This is called *causal masking*.

Thus the attention matrix is forced to have the form

$$w_{ij} = 0 \quad \text{whenever } j > i. \quad (54)$$

This ensures that the representation at position i depends only on the tokens

$$X_1, \dots, X_i. \quad (55)$$

One important computational advantage of transformers is that all these prediction problems can be processed in parallel during training. For example, from the sentence

$$(\text{The, cat, sat, on, the, mat}), \quad (56)$$

we simultaneously obtain the prediction tasks

$$\text{The} \longrightarrow \text{cat}, \quad (57)$$

$$(\text{The, cat}) \longrightarrow \text{sat}, \quad (58)$$

$$(\text{The, cat, sat}) \longrightarrow \text{on}, \quad (59)$$

and so on.

The parameters are learned by stochastic gradient descent or one of its variants, using back-propagation to compute gradients.

3.4 Generating Text and Sampling

After the model has been trained, it can be used to generate text.

Suppose we begin with a prompt

$$X_1, \dots, X_t. \quad (60)$$

The transformer produces the conditional distribution

$$P_\theta(X_{t+1} = v \mid X_1, \dots, X_t), \quad v \in \mathcal{V}. \quad (61)$$

A simple strategy is to choose the token having the largest probability:

$$\hat{X}_{t+1} = \arg \max_{v \in \mathcal{V}} P_\theta(X_{t+1} = v \mid X_1, \dots, X_t). \quad (62)$$

This is called *greedy decoding*. It often produces very predictable text, because the model always chooses the most likely next token.

A more flexible strategy is to sample from the probability distribution

$$P_\theta(X_{t+1} = v \mid X_1, \dots, X_t). \quad (63)$$

Thus tokens with high probability are more likely to be chosen, but less likely tokens may occasionally be chosen as well.

Once a new token has been generated, it is appended to the sequence:

$$X_1, \dots, X_t, \hat{X}_{t+1}. \quad (64)$$

The model is then applied again to obtain

$$P_\theta(X_{t+2} = v \mid X_1, \dots, X_t, \hat{X}_{t+1}), \quad v \in \mathcal{V}. \quad (65)$$

Repeating this procedure generates text one token at a time.

In practice, some of the computation can be reused. Once the key and value vectors for earlier tokens have been computed, they can be stored and reused when generating later tokens. Then, at the next step, the model only needs to process the newly generated token and update the attention calculations involving that token. This reuse of previously computed key and value vectors is often called a *key-value cache*.

Another common sampling method introduces a temperature parameter $\tau > 0$. If $s_t(v)$ denotes the score, or logit, assigned to token v , then instead of using the usual softmax distribution we use

$$p_t^{(\tau)}(v) = \frac{\exp(s_t(v)/\tau)}{\sum_{u \in \mathcal{V}} \exp(s_t(u)/\tau)}. \quad (66)$$

When

$$\tau < 1, \tag{67}$$

the distribution becomes more concentrated on the highest-scoring tokens, and the generated text becomes more deterministic.

When

$$\tau > 1, \tag{68}$$

the distribution becomes flatter, and the generated text becomes more random.

Thus the temperature controls the trade-off between predictability and variety in generated text.

3.5 Word Embeddings and Contextual Representations

Throughout these notes, we have assumed that each token is represented by a vector

$$e_t \in \mathbb{R}^d. \tag{69}$$

A natural question is: where do these vectors come from?

Suppose the vocabulary contains K distinct words. A simple representation of a word is its *one-hot encoding*, namely a vector in $\mathbb{R}^K$ having a single entry equal to one and all remaining entries equal to zero. For example, if a word occupies the third position in the vocabulary, then its one-hot representation is

$$(0, 0, 1, 0, \dots, 0)^T. \tag{70}$$

Although conceptually simple, this representation is unsatisfactory. First, realistic vocabularies may contain hundreds of thousands of words, leading to extremely high-dimensional vectors. Second, one-hot encodings contain no notion of similarity between words. For example, the words “Paris” and “London” are no closer to one another than either is to the word “polynomial”.

To obtain more informative representations, one introduces an embedding matrix

$$E \in \mathbb{R}^{d \times K}, \tag{71}$$

where typically $d \ll K$. Given a one-hot encoded word vector x , its embedding is defined by

$$v = Ex. \tag{72}$$

Because x contains a single nonzero entry, the vector v is simply one column of the matrix E . Thus each word is represented by a vector in a much lower-dimensional space.

A fundamental question is how the embedding matrix E should be chosen. A popular approach, known as *word2vec* (Mikolov et al., 2013), learns embeddings by solving a prediction problem on a large corpus of text.

One version of word2vec, called the skip-gram model, attempts to predict nearby context words from a given central word. In a simplified form, each word w is assigned an embedding vector e_w , and each context word u is assigned a context vector c_u . The model uses the score

$$e_w^T c_u \tag{73}$$

to quantify how likely the word u is to appear near the word w .

Why does this lead to meaningful word representations?

The key observation is that words with similar meanings often occur in similar contexts. For example, the words “Paris” and “London” are likely to occur near words such as

$$\text{city, capital, Europe,} \quad (74)$$

whereas the word “polynomial” typically appears in very different contexts.

Consequently, the model must learn approximately the same predictions for “Paris” and “London”. In particular, for many context words u , the quantities

$$e_{\text{Paris}}^T c_u \quad (75)$$

and

$$e_{\text{London}}^T c_u \quad (76)$$

must be similar.

Equivalently,

$$(e_{\text{Paris}} - e_{\text{London}})^T c_u \approx 0 \quad (77)$$

for a large collection of context vectors c_u . The simplest way for this to happen is for the vectors

$$e_{\text{Paris}} \quad \text{and} \quad e_{\text{London}} \quad (78)$$

to be close together. Thus words that exhibit similar contextual behaviour naturally acquire similar embeddings.

The resulting embedding space often displays a remarkable semantic structure. Words with related meanings tend to cluster together, and many semantic relationships are reflected geometrically in the embedding space.

Historically, word2vec and related methods represented a major advance in natural language processing. However, they have an important limitation: each word is assigned a single fixed vector.

For example, the word “bank” receives the same representation in the sentences

$$\text{“I deposited money in the bank”} \quad (79)$$

and

$$\text{“The boat reached the bank of the river”}. \quad (80)$$

The central innovation of transformers is that they replace these static word representations by *contextual representations*. Instead of assigning a single vector to the word “bank”, the transformer computes a representation whose value depends on the surrounding words. Consequently, the two occurrences of “bank” above receive different vector representations reflecting their different meanings.

In this sense, transformers may be viewed as a natural extension of the ideas underlying word embeddings. Both approaches seek useful numerical representations of words, but transformers go one step further by constructing representations that depend on the entire linguistic context. This ability to build context-dependent representations is one of the key reasons for the success of modern large language models.

3.6 Conclusion

Let us briefly recap the story.

We began with the statistical problem of estimating the conditional probabilities

$$P(X_t \mid X_1, \dots, X_{t-1}), \tag{81}$$

which completely determine the joint distribution of a sequence through the chain rule of probability.

At first sight, this appears to be an impossibly difficult nonparametric estimation problem. The number of possible prefixes grows exponentially with their length, and it is not clear how one could hope to estimate a separate conditional distribution for each possible context.

The central idea underlying modern language models is to replace this enormous collection of conditional distributions by a single parameterized function. The transformer architecture provides a highly flexible family of functions that can act on sequences of arbitrary length. Through repeated applications of attention mechanisms and nonlinear transformations, it converts token embeddings into contextual representations

$$h_1, h_2, \dots, h_t, \tag{82}$$

and then uses these representations to construct probability distributions for the next token.

The model is trained by maximum likelihood estimation on a massive corpus of text. In principle, the objective is extremely simple: predict the next token as accurately as possible. What is remarkable is that solving this seemingly narrow prediction problem appears to require the model to learn much richer structures. In order to predict the next word effectively, the model must discover regularities of grammar, semantics, long-range dependencies, factual knowledge, and many other patterns present in language.

Perhaps the most surprising aspect of modern language models is that none of these concepts are explicitly programmed into the model. The transformer is never directly instructed to learn syntax, semantics, reasoning, or world knowledge. Instead, these capabilities appear to emerge because they are useful for the task of next-token prediction.

At a mathematical level, the internal representations

$$h_1, h_2, \dots, h_t \tag{83}$$

may be viewed as learned summaries of the preceding context. During training, the model discovers representations that retain precisely the information needed for predicting future tokens. The remarkable empirical finding of the last decade is that such representations can be extraordinarily rich when trained on sufficiently large datasets using sufficiently expressive transformer architectures.

Although many theoretical questions remain open, transformers have demonstrated that a relatively simple statistical principle—maximum likelihood estimation applied to next-token prediction—combined with a powerful class of functions can lead to systems exhibiting remarkably sophisticated linguistic behaviour.

In this sense, large language models may be viewed as a striking example of how far classical ideas from probability, statistics, and function approximation can be pushed when combined with modern computing power and large-scale data.

4 Possible Projects

The goal of these projects is not to train a state-of-the-art language model. Modern large language models require enormous datasets and computational resources. Instead, the purpose is to gain a deeper understanding of the ideas discussed in these notes by implementing and experimenting with simplified versions of language models and transformers.

4.1 Project 1: From N-Grams to Language Models

Before the development of neural language models, a common approach was to estimate conditional probabilities using (n)-gram models.

Choose a text corpus, such as Shakespeare, Sherlock Holmes, or a collection of Wikipedia articles. Construct unigram, bigram, and trigram language models and use them to generate text.

Some questions to investigate are:

- How does the quality of generated text change as (n) increases?
- How does the number of parameters grow with (n)?
- What difficulties arise from data sparsity?
- Why might a fixed-order Markov model be inadequate for language?

This project connects directly to the statistical estimation problem discussed in Section 1.

4.2 Project 2: Learning Word Embeddings

Implement a simplified version of the word2vec algorithm.

Using a text corpus, learn embedding vectors for words and visualize them using principal component analysis (PCA) or another dimensionality reduction method.

Questions to investigate include:

- Do semantically related words cluster together?
- Which words are nearest neighbours of words such as ‘Paris’, ‘London’, ‘cat’, or ‘dog’?
- How does the embedding dimension affect the results?
- Does the learned geometry appear to capture semantic information?

This project provides hands-on experience with representation learning.

4.3 Project 3: Visualizing Attention

Using a pretrained transformer model, examine the attention weights produced by different attention heads.

For example, consider sentences involving pronouns or long-range dependencies:

“The animal did not cross the street because it was tired.” (84)

or

“The trophy does not fit in the suitcase because it is too large.” (85)

Visualize the attention matrices and investigate which words attend to which other words. Questions to explore include:

- Do different attention heads learn different types of relationships?
- Can attention identify pronoun references?
- What linguistic structures appear to be captured?

4.4 Project 4: Building a Tiny Transformer

Implement a small transformer language model from scratch.

The model need only contain:

- token embeddings,
- positional encodings,
- one or two attention layers,
- feedforward nonlinearities,
- a softmax output layer.

Train the model on a small text corpus and monitor the training loss.

Possible experiments include:

- removing positional encodings,
- reducing the number of attention heads,
- changing the embedding dimension,
- varying the number of transformer layers.

This project provides direct experience with the transformer architecture described in these notes.

4.5 Project 5: Sampling and Temperature

Using a pretrained language model, investigate the effect of different sampling strategies.

Generate text using:

- greedy decoding,
- sampling from the softmax distribution,
- temperature values ($\tau = 0.2, 0.5, 1, 2$).

Questions to investigate include:

- How does the generated text change as the temperature varies?

- When does the output become repetitive?
- When does the output become incoherent?
- How does randomness affect creativity?

This project illustrates the ideas discussed in Section 3.4.

4.6 Project 6: Word2Vec versus Transformers

One of the most important conceptual advances of transformers is the transition from static embeddings to contextual representations.

Consider the sentences

“I deposited money in the bank.” (86)

and

“The boat reached the bank of the river.” (87)

Using a word2vec model, compute the embedding of the word “bank” in both sentences.

Then use a transformer model such as BERT and extract the contextual representation of “bank” in the two contexts.

Compare the resulting vectors and discuss the differences.

Questions to investigate include:

- Why are the word2vec representations identical?
- Why are the transformer representations different?
- What does this reveal about contextual representations?

This project directly illustrates one of the central ideas behind modern transformer architectures.

4.7 Project 7: Does Attention Learn Grammar?

Using a pretrained transformer model, investigate whether attention weights capture grammatical structure.

Possible questions include:

- Does attention connect pronouns with their antecedents?
- Does attention identify subject–verb relationships?
- Are long-range dependencies visible in the attention patterns?
- Do different layers appear to encode different kinds of information?

This project explores the extent to which linguistic structure emerges from the next-token prediction objective.

4.8 Project 8: Reproducing the Transformer Pipeline

Create an interactive notebook explaining and illustrating the complete pipeline discussed in these notes:

1. tokenization,
2. word embeddings,
3. positional encoding,
4. attention,
5. multi-head attention,
6. feedforward nonlinearities,
7. next-token prediction,
8. sampling.

The notebook should contain visualizations, examples, and numerical experiments designed to explain each component.

This project is particularly suitable for students interested in teaching, scientific communication, or expository writing.

Among these projects, Projects 2, 3, and 6 are particularly recommended. Together they illustrate the progression from word embeddings to attention mechanisms and finally to contextual representations, which is one of the central themes of modern natural language processing.